NAME:J.A.MAANSI

REG NO:22MIC0055

TASK 4

[7]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, precision_score, recall_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt

try:
    data = pd.read_csv(r"C:\Users\julug\Downloads\dataset.csv")
    print("Dataset loaded successfully!")
    print("Columns in dataset:", data.columns.tolist())
except FileNotFoundError:
    print("Error: File not found. Please check the file path.")
    exit()
except Exception as e:
    print(f"Error loading dataset: {e}")
    exit()

if 'id' in data.columns:
    data = data.drop(columns=['id'])

data = data.dropna(axis=1, how='all')

if 'diagnosis' not in data.columns:
    print("Error: 'diagnosis' column not found in dataset")
    exit()

data['diagnosis'] = data['diagnosis'].map({'M': 1, 'B': 0})

X = data.drop(columns=['diagnosis'])
y = data['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

y_probs = model.predict_proba(X_test_scaled)[:, 1]
y_pred = (y_probs >= 0.5).astype(int)

cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_probs)

print("\nModel Evaluation:")
print("Confusion Matrix:\n", cm)
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"ROC-AUC Score: {roc_auc:.4f}")

fpr, tpr, thresholds = roc_curve(y_test, y_probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.2f})")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid()
plt.show()

thresholds_to_try = [0.3, 0.4, 0.6, 0.7]
print("\nPerformance at Different Thresholds:")
for threshold in thresholds_to_try:
    y_pred_thresh = (y_probs >= threshold).astype(int)
    print(f"\nThreshold: {threshold}")
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_thresh))
    print(f"Precision: {precision_score(y_test, y_pred_thresh):.4f}")
    print(f"Recall: {recall_score(y_test, y_pred_thresh):.4f}")
```

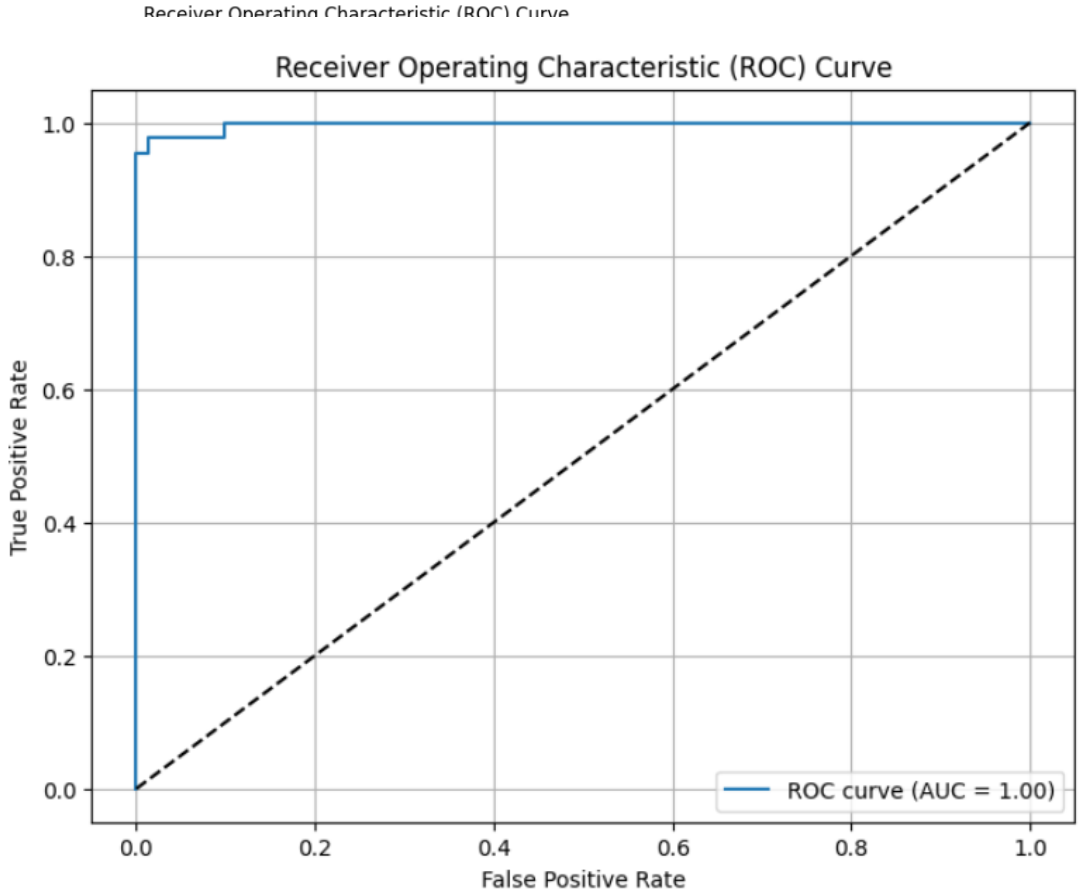
```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print("\nSigmoid Function Explanation:")
print("The sigmoid function maps any real number to a probability between 0 and 1.")
print("Examples:")
print(f"sigmoid(0) = {sigmoid(0):.4f}")
print(f"sigmoid(2) = {sigmoid(2):.4f}")
print(f"sigmoid(-2) = {sigmoid(-2):.4f}")
```

OUTPUT:

```
Dataset loaded successfully!
Columns in dataset: ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_m
ean', 'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean', 'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se', 'compactne
ss_se', 'concavity_se', 'concave points_se', 'symmetry_se', 'fractal_dimension_se', 'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst', 'sm
oothness_worst', 'compactness_worst', 'concavity_worst', 'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst']

Model Evaluation:
Confusion Matrix:
 [[70  1]
 [ 2 41]]
Precision: 0.9762
Recall: 0.9535
ROC-AUC Score: 0.9974
```



```
Performance at Different Thresholds:

Threshold: 0.3
Confusion Matrix:
 [[67  4]
 [ 1 42]]
Precision: 0.9130
Recall: 0.9767

Threshold: 0.4
Confusion Matrix:
 [[70  1]
 [ 1 42]]
Precision: 0.9767
Recall: 0.9767

Threshold: 0.6
Confusion Matrix:
 [[71  0]
 [ 2 41]]
Precision: 1.0000
Recall: 0.9535

Threshold: 0.7
Confusion Matrix:
 [[71  0]
 [ 2 41]]
Precision: 1.0000
Recall: 0.9535

Sigmoid Function Explanation:
The sigmoid function maps any real number to a probability between 0 and 1.
Examples:
sigmoid(0) = 0.5000
sigmoid(2) = 0.8808
sigmoid(-2) = 0.1192
```